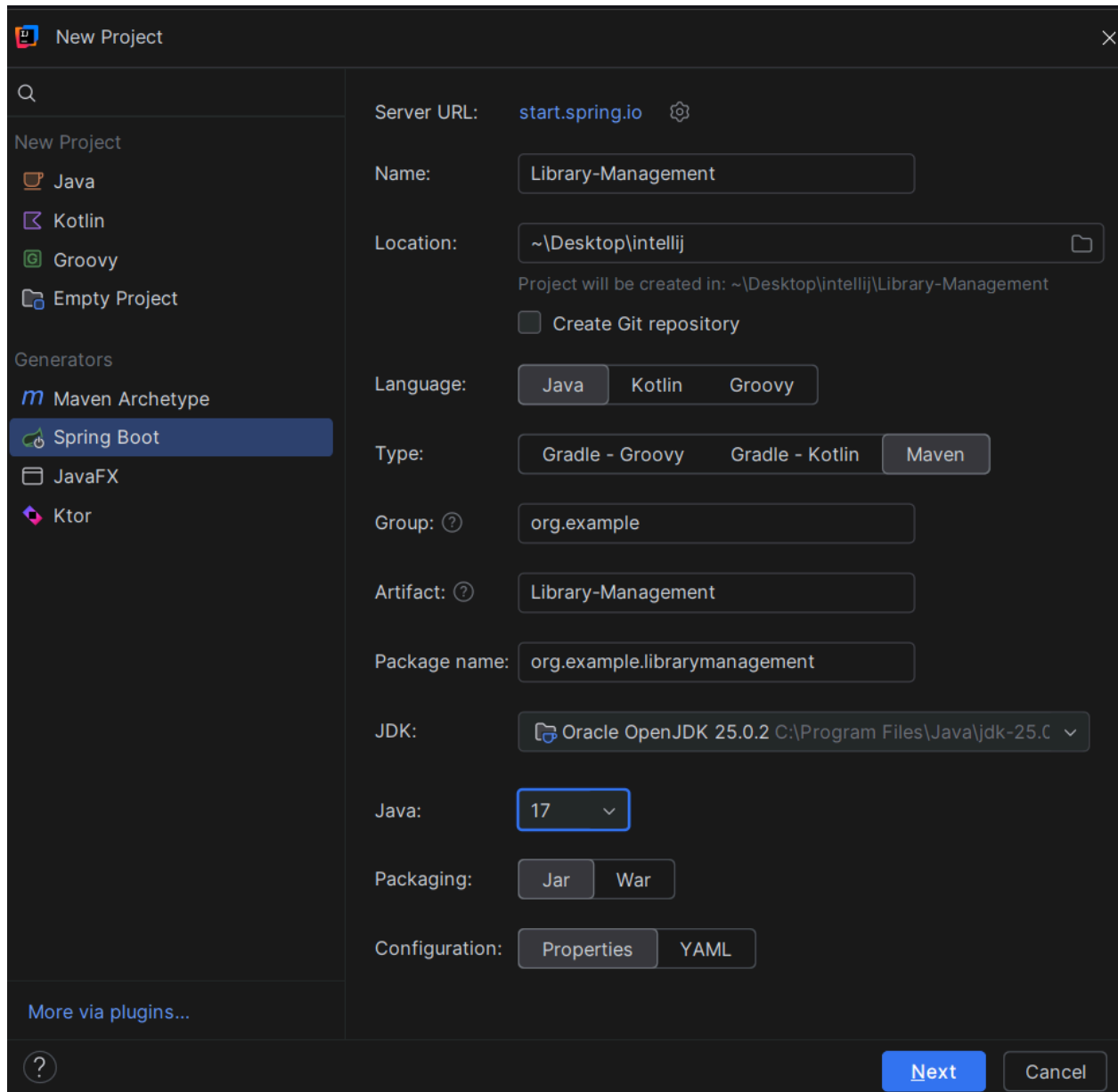


Exercise 9: Creating a Spring Boot Application

Project Creation:



The image shows the 'New Project' dialog in IntelliJ IDEA. The left sidebar lists project types: 'New Project' (Java, Kotlin, Groovy, Empty Project) and 'Generators' (Maven Archetype, Spring Boot, JavaFX, Ktor). 'Spring Boot' is selected. The main area contains configuration fields: 'Server URL' (start.spring.io), 'Name' (Library-Management), 'Location' (~\Desktop\intellij), 'Language' (Java), 'Type' (Maven), 'Group' (org.example), 'Artifact' (Library-Management), 'Package name' (org.example.librarymanagement), 'JDK' (Oracle OpenJDK 25.0.2), 'Java' (17), 'Packaging' (Jar), and 'Configuration' (Properties). A 'Next' button is at the bottom right.

New Project

Search

New Project

- Java
- Kotlin
- Groovy
- Empty Project

Generators

- Maven Archetype
- Spring Boot**
- JavaFX
- Ktor

More via plugins...

Server URL: start.spring.io

Name: Library-Management

Location: ~\Desktop\intellij

Project will be created in: ~\Desktop\intellij\Library-Management

☐ Create Git repository

Language: Java Kotlin Groovy

Type: Gradle - Groovy Gradle - Kotlin **Maven**

Group: ? org.example

Artifact: ? Library-Management

Package name: org.example.librarymanagement

JDK: Oracle OpenJDK 25.0.2 C:\Program Files\Java\jdk-25.0

Java: 17

Packaging: Jar War

Configuration: Properties **YAML**

Next Cancel

File Structure:



Library-Management ▾

Version Control ▾

Project ▾

Structure Alt+7

> .idea

> .mvn

▾ src

▾ main

▾ java

▾ com.library

> controller

> entity

> repository

Main

▾ org.example.librarymanagement

LibraryManagementApplication

▾ resources

static

templates

application.properties

▾ test

▾ java

▾ org.example.librarymanagement

LibraryManagementApplicationTests

> target

.gitattributes

.gitignore

[M↓ HELP.md](#)

Code:

Main.java:

```
package com.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Main {

    public static void main(String[] args) {
        SpringApplication.run(Main.class, args);
    }

}
```

Application.properties:

```
spring.application.name=LibraryManagement

spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.h2.console.enabled=true
```

Book Controller.java

```
package com.library.controller;

import com.library.entity.Book;
import com.library.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/books")
public class BookController {
```

```

    @Autowired
    private BookRepository repository;

    @GetMapping
    public List<Book> getBooks() {
        return repository.findAll();
    }

    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return repository.save(book);
    }
}

```

Book.java

```

package com.library.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Book {

    @Id
    private int id;
    private String title;
    private String author;

    public Book() {
    }

    public Book(int id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }
}

```

```
public void setTitle(String title) {  
    this.title = title;  
}  
  
public String getAuthor() {  
    return author;  
}  
  
public void setAuthor(String author) {  
    this.author = author;  
}  
}
```

BookRepository.java

```
package com.library.repository;  
  
import com.library.entity.Book;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface BookRepository extends JpaRepository<Book, Integer> {  
  
}
```

OUTPUT:

History

NewImport

POST http://localhost:8080/b...

POST http://localhost:8080/books

POST http://localhost:8080/books

POST POSThttp://localhost:8080/books http://localhost...

POST POST http://localhost:8080/books

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

POST http://localhost:8080/b...

POST http://localhost:8080/books

ParamsAuthorizationHeaders (9)BodyPre-request ScriptTestsSettings

noneform-datax-www-form-urlencodedrawbinaryJSON

```
1 {
2   "id": 1,
3   "title": "Spring Boot",
4   "author": "John"
5 }
```

BodyCookiesHeaders (5)Test Results

Status: 200 OKTime: 515 msSize: 202 BSave Response

PrettyRawPreviewVisualizeJSON

```
1 {
2   "id": 1,
3   "title": "Spring Boot",
4   "author": "John"
5 }
```

Console

Not connected to a Postman account

2 ErrorsAll LogsClear

POST post http://localhost:8080/books

POST posthttp://localhost:8080/books90Ahttp://localhost:8080/books

POST http://localhost:8080/books400617 ms

POST http://localhost:8080/books200515 ms